

**FINAL PROJECT: FUNDAMENTALS
OF ARTIFICIAL INTELLIGENCE**

ACADEMIC YEAR 2025-2026

"SEARCHING FOR COLONEL KURTZ"

Claudia Moya Rodríguez

Madrid, January 2026

PART 1

1. Objective and solution approach

The first part implements the palace environment as a **6 x 6 grid** (36 cells). Captain Willard always enters at cell (1, 1) and must first **find Colonel Kurtz** and then **leave through the exit**, while avoiding three pits and a hostile soldier. The key condition is that the agent **does not know the real map**: it can reason only from the history of visited cells and the percepts received during exploration. From this accumulated information, it must determine which adjacent cells are **safe**, meaning that it can prove they contain neither a pit nor the soldier.

The solution is implemented as a **user-guided logical agent**: the user chooses each movement with the WASD keys, while the program maintains a **knowledge base (KB)**, updates its deductions on every turn, and prints both the knowledge board and movement recommendations. An automatic mode and a planner with search traces are included as extensions.

2. Hidden world and game dynamics

The real world is generated by placing three pits, one soldier, one exit, and Kurtz in mutually distinct cells that are also different from the start. The default board size is fixed at six, and the program verifies that enough cells exist before sampling all elements without overlap. Generation is random, so the initial cell may legitimately have a breeze or glow; the assignment imposes no additional constraint around (1, 1).

An optional integer seed makes an experiment reproducible. Entering the same seed recreates the same placement of pits, soldier, exit, and Kurtz, which is useful for debugging, comparing strategies, and documenting a concrete example.

The dynamic state stores the agent's position; whether the agent and soldier are alive; whether Kurtz has been found; whether the grenade remains available; a one-turn scream flag; and whether the exit has been visited. Movement attempts to move one cell in one of the four cardinal directions. A wall collision leaves the position unchanged. After a valid move, the program checks for death and updates the Kurtz and exit events. Once Kurtz is found, he travels with the captain and the displayed position changes accordingly.

The **grenade** is an additional one-use action aimed at an adjacent cell. If it hits the live soldier, the soldier dies and the next percept contains `Scream = 1`. That signal is consumed by the following percept, so it cannot remain active indefinitely.

3. Percepts: information received by the captain

The percept at state s is represented as a Boolean list in the exact order defined by the assignment:

`percept(s) = [Breeze, Snore, Glow, WallUp, WallDown, WallLeft, WallRight, Scream]`

Breeze is 1 when an orthogonally adjacent cell contains a pit. **Snore** is 1 when the live soldier is adjacent. **Glow** is 1 when the exit is in the current cell or an adjacent cell. The four wall signals identify the board boundary. **Scream** is 1 only on the turn immediately following a successful grenade hit. The program computes the percept internally in this order but prints it in a more readable labelled form without changing its meaning.

4. Knowledge base: storage and update

The KB represents the captain's mental state: everything known or believed from observations. On each turn, after reading the current cell's percept, it stores the observation and recalculates global inferences. It explicitly maintains the visited set and four persistent dictionaries of observations, so knowledge accumulates instead of depending only on the latest percept.

The KB also stores three main inferred structures:

1. A list of possible pit worlds compatible with all observations.
2. The set of candidate cells for the soldier.
3. The set of candidate cells for the exit. When a candidate set is reduced to one cell, that certainty is also stored explicitly.

Inference is repeated a small number of times because the deductions depend on one another. For example, a cell proved to contain the soldier or exit must be excluded from the pit configurations. Iteration propagates these cross-constraints within the same turn and prevents inconsistent hypotheses from being displayed.

5. Pit deduction through consistent worlds

The pit model deliberately uses a strong method rather than isolated local rules: it enumerates complete configurations containing exactly three pits and retains only those that satisfy every breeze observation collected so far.

A visited cell with Breeze = 0 rules out pits in all orthogonal neighbours. A visited cell with Breeze = 1 requires at least one neighbouring pit. Visited cells and cells already known to contain incompatible entities are excluded. The result is the list *pit_worlds*, whose size also provides a useful measure of how exploration reduces uncertainty.

This representation supports two important deductions. A cell is a certain pit if it appears in every consistent world, and it is a possible pit if it appears in at least one. Most importantly for action selection, a cell is proved safe from pits only if it appears in none of the consistent worlds. If no consistent worlds exist, the system does not claim safety from missing evidence.

6. Soldier and exit inference

The soldier is inferred with a candidate set. Every positive snore restricts the candidate location to the orthogonal neighbours of that observation, while every negative snore removes those neighbours. Visited cells and confirmed pit cells are also discarded. If the candidate set is reduced to one cell, the soldier is considered known. Once the soldier is dead, its candidate set is cleared.

The exit is handled similarly, with one important distinction: glow covers the current cell as well as its orthogonal neighbours. Positive glow intersects the exit candidates with that area, and negative glow removes it. The code also distinguishes a logically inferred candidate from a physically discovered exit, which is confirmed only after the agent enters its cell.

7. Safe cells and movement recommendations

The implementation follows the assignment's strict definition. A visited cell is safe because the captain has already survived there. An unvisited cell is safe only when the KB proves both that it is absent from every consistent pit world and, while the soldier is alive, that it is not a soldier candidate. Movement recommendations are therefore based on demonstrable safety, not on access to the hidden map.

8. Knowledge-board visualisation (not the real map)

The display shows the agent's knowledge rather than the real map. The captain is represented as CW, or CWK after finding Kurtz, with suffixes for relevant percepts. Visited cells, demonstrably safe cells, and hypotheses about hazards or objectives on the frontier use distinct markers. A fixed width per cell keeps the grid aligned even when combined labels become longer.

Alongside the board, the program prints the dynamic state, the decomposed percept, and an uncertainty summary: the number of consistent pit worlds and the candidate sets for the soldier and exit. These diagnostics make every belief and recommendation inspectable.

9. Planning with BFS, DFS, GBFS, and A*

The planner supports breadth-first search (BFS), depth-first search (DFS), Greedy Best-First Search (GBFS), and A*. An optional trace displays the frontier, removed node, and explored set at every step. BFS and DFS report accumulated unit cost g ; GBFS reports the heuristic h ; and A* reports $f = g + h$.

GBFS and A* use Manhattan distance:

$$h((r, c)) = |r - r_{\text{goal}}| + |c - c_{\text{goal}}|$$

This heuristic is admissible for four-connected, unit-cost movement because each action changes only one coordinate by one unit, so no route can require fewer moves than the sum of the row and column differences.

$$h(n) \leq \text{optimal cost from } n \text{ to the goal}$$

It is also consistent: for neighbouring states n and n' , one move costs 1 and Manhattan distance changes by at most 1, which gives $h(n) \leq 1 + h(n')$. Therefore an A* node does not need a later lower-cost reopening under these movement assumptions.

A separate function supplies the goal heuristic instead of revealing a hidden objective directly to the planner. The goal test is expressed as zero heuristic distance. This keeps the planner reusable when the objective changes from Kurtz to the exit.

10. Two-stage automatic mode and explicit risk management

The automatic extension plans the complete mission rather than selecting a single locally attractive move. It first plans a route to Kurtz and then, once he has been rescued, a route to the exit. The search state is rebuilt between phases while the accumulated knowledge remains available, so the agent changes objective without forgetting its observations.

The conservative rule is to generate movements only towards cells proved safe by the KB. If uncertainty leaves no complete safe plan, the program can explicitly offer a controlled risk decision instead of silently describing a speculative move as safe.

Planning also uses a side-effect-free simulated percept function. Hypothetical evaluation must not alter the real world or consume the temporary scream signal; separating simulation from execution avoids unintended state changes.

11. Verification against the assignment

The implementation preserves the required structure: a 6 x 6 board; exactly three pits; one soldier, one exit, and Kurtz in distinct cells; the specified percept order; persistent knowledge; and the strict safety definition. The visible board represents the knowledge state, while the hidden placement is used only by the environment to produce percepts and resolve actions.

PART 2

Bayesian Palace

This version places the agent in an uncertain environment where the locations of traps, the soldier, the exit, and Kurtz are not known with certainty. Instead of deterministic conclusions alone, the system maintains probability distributions that are updated from percepts. These posterior beliefs become numerical risk maps that guide decision-making.

1. Belief representation

For each entity τ in $\{F, P, D, M, S, CK\}$, a discrete distribution is defined over the grid:

$$\text{Bel}_{\tau}(i, j) = P(\text{entity } \tau \text{ is located at cell } (i, j))$$

Every distribution is non-negative and normalised:

$$\text{Bel}_{\tau}(i, j) \geq 0, \quad \sum_{i,j} \text{Bel}_{\tau}(i, j) = 1$$

2. Stimulus zone and percepts

A stimulus generated by an entity at cell c is perceived in that cell and in its orthogonal neighbours. This region is the stimulus zone:

$$Z(c) = \{c\} \cup \text{Neighbours}_4(c)$$

If the agent is at the current cell x , the binary percept e_{τ} indicates whether entity τ lies in that zone.

3. Bayesian update with deterministic likelihood

The likelihood is binary: a candidate location is either compatible or incompatible with the observed percept.

$$L(e_{\tau} \mid c) = 1 \text{ if } [x \in Z(c)] \text{ is equivalent to } [e_{\tau} = 1]; \text{ otherwise } 0$$

The previous belief is multiplied by this likelihood and then renormalised:

$$\text{Bel}'_{\tau}(c) = \text{Bel}_{\tau}(c)L(e_{\tau} \mid c) / \sum_{c'} \text{Bel}_{\tau}(c')L(e_{\tau} \mid c')$$

Visited cells are removed as possible locations for traps and the live soldier. The Kurtz posterior is also updated when he is rescued, and the exit posterior becomes certain once the exit is physically discovered.

4. Structural constraint: entities cannot occupy traps

The soldier, exit, and Kurtz can be generated only in cells free from all traps. For a particular trap configuration (F, P, D), define:

$$\text{Safe}(F, P, D) = \{c \mid c \notin (F \cup P \cup D)\}$$

The probability of a safe entity occupying cell c , conditioned on that configuration, is uniform over the safe set:

$$P(c \mid F, P, D) = 1 / |\text{Safe}(F, P, D)| \text{ if } c \in \text{Safe}(F, P, D); \text{ otherwise } 0$$

A safety-induced prior is obtained by averaging over possible trap configurations:

$$w(c) = E_{F,P,D}[1(c \in \text{Safe}(F,P,D)) / |\text{Safe}(F,P,D)|]$$

This prior is combined with the evidence-based posterior for the soldier, exit, and Kurtz. The exact calculation is cached because repeated enumeration of every trap triple is one of the most expensive operations in the Bayesian agent.

5. Probability of at least one trap

Assuming independence between the three trap-type marginals, the probability that cell c contains at least one trap is:

$$P(F \cup P \cup D \mid c) = 1 - (1 - P_F(c))(1 - P_P(c))(1 - P_D(c))$$

6. Risk of death on entering a cell

Because the soldier and traps cannot coexist in the generated world, the total estimated probability of death is:

$$P(\text{death} \mid c) = \min(1, P(F \cup P \cup D \mid c) + P_M(c))$$

7. Risk threshold and planning

A cell is available to the planner when its estimated death probability is strictly below a configurable threshold p :

$$c \text{ is permitted} \Leftrightarrow P(\text{death} \mid c) < p$$

8. Permitted-cell set

$$C_p = \{c \mid P(\text{death} \mid c) < p\}$$

The default threshold is 0.20. Lower values produce more conservative behaviour, while higher values open more routes at the cost of greater estimated danger. Visited cells and a discovered exit remain available so the planning graph stays usable.

River Crossing as an MDP

The river-crossing problem is formulated as a Markov decision process (MDP). Starting on the left bank, Captain Willard and Kurtz must reach an exit on the right bank while maximising expected discounted reward. Movement is uncertain because interior river columns apply a stochastic downward current.

1. Formal definition of the MDP

A Markov decision process is defined by the tuple

$$(S, A, P, r, \gamma)$$

1.1 States (S)

A state is the agent's discrete position in a two-dimensional grid:

$$s = (i, j), \quad i \in \{0, \dots, \text{nrows} - 1\}, \quad j \in \{0, \dots, \text{ncols} - 1\}$$

Cells labelled as islands are physical obstacles and are not reachable states. Any transition that would leave the board or enter an island is redirected to the current state. Islands limit movement but do not carry a separate penalty beyond the ordinary step cost.

1.2 Actions (A)

The available actions are

$$A = \{\text{up}, \text{down}, \text{left}, \text{right}, \text{stay}\}$$

The *stay* action allows the policy to wait in the current cell, although it may still be affected by the current. The *down* action is implemented as a deterministic downward move.

2. Dynamics and transition probability $P(s' | s, a)$

The transition model is stochastic for actions other than *down*. In the two bank columns the current is zero. In each interior column, a current strength q is sampled between 0.06 and 0.94. The intended successor occurs with probability $1 - q$, while a downward drift occurs with probability q . If both outcomes coincide, their probabilities are merged. Invalid destinations become self-loops, and terminal states remain fixed with probability one.

3. Rewards $r(s, a, s')$

Every executed action has a base cost of -1, including *stay*. Entering the exit adds 100, so the successful transition has a net reward of 99:

$$r(s, a, s') = -1 + 100 \cdot 1[s' = \text{Exit}]$$

A blocked attempt simply receives the standard -1 cost. This encourages short trajectories without treating islands as failure states.

4. Optional extension: hazardous cells (P)

The environment can include traversable hazardous cells. Entering one applies a configurable penalty, -100 by default:

$$r(s, a, s') = -1 + 100 \cdot 1[s' = \text{Exit}] - 100 \cdot 1[s' \in \text{Hazard}]$$

Hazards can be configured as terminal or non-terminal, allowing the same implementation to represent fatal states or severe but recoverable setbacks.

5. Terminal states

The exit is always terminal. When terminal hazards are enabled, a hazard also ends the episode. Every terminal state transitions to itself with probability one and is assigned no further action value.

6. Solution by Value Iteration

Value Iteration begins with zero value for every valid state and repeatedly applies the Bellman optimality update until the largest change falls below 10^{-6} or the iteration limit is reached:

$$V_{k+1}(s) = \max_{a \in A} \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V_k(s')]$$

The executable uses $\gamma = 0.95$. After convergence, the greedy optimal policy is extracted from the final value function:

$$\pi(s) = \arg \max_{a \in A} \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V(s')]$$

7. Simulation and empirical validation

After computing the policy, the program simulates one complete episode from the start. At each state, it selects the policy action and samples the successor from the transition distribution:

$$s' \sim P(\text{next state} \mid s, \pi(s))$$

The trace records the state, action, sampled successor, transition cause, immediate reward, and accumulated return. A fixed seed reproduces both environment generation and the sampled episode. One trajectory illustrates behaviour; a stronger empirical evaluation would repeat the simulation and report success rate, return distribution, and path length.

8. Relationship to heuristic search

Manhattan distance is analysed in this project because it guides GBFS and A* in the palace planners. It is **not used by the river's Value Iteration algorithm**, which evaluates the complete stochastic transition and reward model. Keeping these roles separate avoids conflating heuristic graph search with dynamic programming.

9. Final considerations

The three implementations form a progression from logical certainty to probabilistic belief and stochastic control. Their main internal products - consistent worlds, candidate sets, posterior distributions, risk maps, search traces, values, policies, and simulated transitions - are exposed in the console so that decisions can be inspected rather than treated as a black box.

The programs require only the Python standard library, accept optional seeds for reproducibility, and are accompanied by dependency-free regression tests. The source code remains in Spanish as originally submitted, while this English report and the repository README make the work accessible to an international audience.